Implement a chat client server program using socket programming

Exp.no: 12.b

Date: 26/09/2026

Aim:

To implement a chat client server program using socket programming that enables communication between a client and a server through a network connection.

Algorithm:

Server side:

1.) Create a socket using the socket library.

2.) Bind the socket to an Ip address and port number.

3.) Listen for incoming client connection requests.

4.) Accept the client connection

5.) Receive messages from the client

6.) Send a response message to the client.

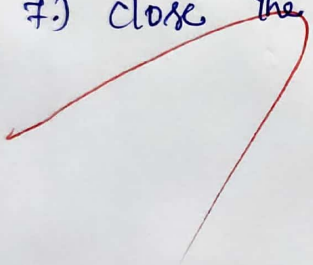
7.) Repeat the send and receive process until the chat is terminated.

8.) Close the connection and socket.

Scanned with OKEN Scanner

Client Side:

1) Create a socket using the socket library.
2) Connect the socket to the server using the its Ip address and port number.
3) Send a message to the server.
4) Receive & the server's response.
5) Repeat the send and receive process for continuous communication.
6) Terminate the chat when required.
7) close the socket connection.



Chat client and Server program :

Chat Client

```java
import java.io.*;
import java.net.*;

public class ChatClient {

    private static final String SERVER_ADDRESS = "127.0.0.1";
    private static final int SERVER_PORT = 5000;

    Run | Debug
    public static void main(String[] args) throws IOException {
        Socket socket = new Socket(SERVER_ADDRESS, SERVER_PORT);
        BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
        PrintWriter out = new PrintWriter(socket.getOutputStream(), autoFlush: true);
        BufferedReader console = new BufferedReader(new InputStreamReader(System.in));

        Thread listener = new Thread(() -> {
            String serverMessage;
            try {
                while ((serverMessage = in.readLine()) != null) {
                    System.out.println(serverMessage);
                }
            } catch (IOException e) {
                System.out.println(x: "Disconnected from server.");
            }
        });
        listener.setDaemon(on: true);
        listener.start();

        String userInput;
        while ((userInput = console.readLine()) != null) {
            out.println(userInput);
            if (userInput.equalsIgnoreCase(anotherString: "/quit")) {
                break;
            }
        }

        socket.close();
    }
}
```

Chat Server

```java
import java.io.*;
import java.net.*;
import java.util.*;

public class ChatServer {

    private static final int PORT = 5000;
    private static final Set<ClientHandler> clients = Collections.synchronizedSet(new HashSet<>());

    Run | Debug
    public static void main(String[] args) throws IOException {
        ServerSocket serverSocket = new ServerSocket(PORT);
        System.out.println("Chat server started on port " + PORT);

        while (true) {
            Socket socket = serverSocket.accept();
            ClientHandler handler = new ClientHandler(socket);
            clients.add(handler);
            new Thread(handler).start();
        }
    }

    private static void broadcast(String message, ClientHandler sender) {
        synchronized (clients) {
            for (ClientHandler client : clients) {
                if (client != sender) {
                    client.send(message);
                }
            }
        }
    }

    private static void removeClient(ClientHandler handler) {
        clients.remove(handler);
        broadcast(handler.getName() + " has left the chat.", handler);
        System.out.println(handler.getName() + " disconnected.");
    }

    private static class ClientHandler implements Runnable {
        private final Socket socket;
        private BufferedReader in;
        private PrintWriter out;
        private String name;
```

Scanned with OKEN Scanner

```
    ClientHandler(Socket socket) {
        this.socket = socket;
    }

    String getName() {
        return name;
    }

    void send(String message) {
        out.println(message);
    }

    @Override
    public void run() {
        try {
            in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
            out = new PrintWriter(socket.getOutputStream(), autoFlush: true);

            out.println(x: "Enter your name:");
            name = in.readLine();
            if (name == null) {
                name = "Anonymous-" + socket.getPort();
            }

            System.out.println(name + " connected from " + socket.getInetAddress());
            broadcast(name + " has joined the chat.", this);
            out.println("Welcome, " + name + "! Type /quit to exit.");

            String line;
            while ((line = in.readLine()) != null) {
                if (line.equalsIgnoreCase(anotherString: "/quit")) {
                    break;
                }
                System.out.println(name + ": " + line);
                broadcast(name + ": " + line, this);
            }
        } catch (IOException e) {
            System.out.println("Connection error with " + name + ": " + e.getMessage());
        } finally {
            closeConnection();
        }
    }

    private void closeConnection() {
        try {
            removeClient(this);
            socket.close();
        } catch (IOException e) {
            // ignore
        }
    }
}
}
```

Scanned with OKEN Scanner

Output :

CHAT CLIENT

```
devyesh@Macs-MacBook-Air Downloads % java ChatClient
Enter your name:
Devyesh
Welcome, Devyesh! Type /quit to exit.
Hello
I am Devyesh
Nice to meet you
Bye!
/quit
Disconnected from server.
```

CHAT SERVER

```
devyesh@Macs-MacBook-Air Downloads % javac ChatServer.java
devyesh@Macs-MacBook-Air Downloads % java ChatServer
Chat server started on port 5000
Devyesh connected from /127.0.0.1
Devyesh: Hello
Devyesh: I am Devyesh
Devyesh: Nice to meet you
Devyesh: Bye!
Devyesh disconnected.
```

Result :

Thus, the chat client & server program using socket programming was successfully implemented and the output was verified.

Scanned with OKEN Scanner